

A NEW ARCHITECTURE OF HIGH-ORDER DEEP NEURAL NETWORKS THAT LEARN MARTINGALES

YUMING MA² AND SYOITI NINOMIYA¹

ABSTRACT. A new deep-learning neural network architecture based on high-order weak approximation algorithms for stochastic differential equations (SDEs) is proposed. The architecture enables the efficient learning of martingales by deep learning models. The behaviour of deep neural networks based on this architecture, when applied to the problem of pricing financial derivatives, is also examined. The core of this new architecture lies in the high-order weak approximation algorithms of the explicit Runge–Kutta type, wherein the approximation is realised solely through iterative compositions and linear combinations of vector fields of the target SDEs.

1. INTRODUCTION AND BACKGROUND

The objective of the present paper is to propose a new class of deep neural network architectures for learning martingales, or, more specifically, the vector fields that determine the stochastic differential equations that represents these martingales. The architecture that we present here has two main features. The initial feature is that the network learns the coefficient functions, i.e. vector fields, that are contained within the stochastic differential equations that describe the diffusion process of the target that is to be learned. The second feature is that it is based on a network that represents a high-order weak approximation method for stochastic differential equations. The following sections provide an overview and background explanation for each of these two features. Henceforth, deep neural networks will be referred to as DNNs, neural networks as NNs, and stochastic differential equations will be abbreviated as SDEs.

1.1. Deep neural network machines for learning diffusion processes. DNN learning machines are a class of learning architectures that have achieved remarkable success, beginning with pioneering work such as [1], [7], and [18]. A DNN consists of numerous components, known as layers, arranged in a serial configuration. Each layer receives a finite-dimensional vector as input and produces a finite-dimensional output vector, i.e. it implements a mapping from $\mathbb{R}^N$ to $\mathbb{R}^M$. A common design involves composing a linear transformation from $\mathbb{R}^N$ to $\mathbb{R}^M$ with a simple nonlinear function, referred to as an activation function. For simplicity, it is assumed in the following that both the input and output dimensions of each layer are equal to N . A learning machine composed of multiple such layers is referred to as a multilayer neural network (MNN), and when the number of layers is large, the architecture is termed a DNN.

Among the many important discoveries in the progress of DNN learning machines, the following three works are most relevant to this present paper. The first is [10], which proposes the so-called ResNet. The second is [6], which proposes to

regard the concept of ResNet as a numerical solver of ordinary differential equations. The third is [4], who extends this proposal to the case of SDEs and proposes a DNN learning machine to learn stochastic processes, which is successfully applied to hedging methods in mathematical finance.

1.2. High-order weak approximation algorithms for SDEs. This section introduces two high-order discretisation methods for weak approximation of SDEs, as proposed in [29] and [23]. These two methods have one notable feature in common. This is that they can be implemented in the same way as the explicit Runge–Kutta method, i.e. they can be implemented using only two types of operations: linear combination of real vectors and function application.

It is a natural approach to consider high-order discrete approximations of SDEs using Itô–Taylor expansions. A series of studies on this topic, initiated by [22], has been widely referenced in numerous texts, including [15]. However, the application and study of these approaches has been limited due to the difficulty of dealing with iterative integrals of Brownian motion in multidimensional cases, i.e. $d \geq 2$, in the notation below, and the immaturity of high-dimensional numerical integration techniques.

Quasi-Monte Carlo methods are subsequently found by [28] to be effective for high-dimensional numerical integrals in the weak approximation of SDEs. Then in [16, 17] and [20] these difficulties are overcome and the theory of high-order discretisation of SDEs is established.

1.2.1. Notation and conventions associated with SDEs. Let $(\Omega, \mathcal{F}, P)$ be a probability space, $(B^1(t) \dots B^d(t))$ be the d -dimensional standard Brownian motion, $B^0(t) = t$, and $C_b^\infty(\mathbb{R}^N; \mathbb{R}^N)$ be the set of $\mathbb{R}^N$ -valued smooth functions defined in $\mathbb{R}^N$ whose derivatives of any order are bounded. I_N denotes the identity map on $\mathbb{R}^N$. We also let $X(t, x)$ denote a $\mathbb{R}^N$ -valued diffusion process defined by the stochastic differential equation (SDE) [13]:

$$(1) \quad X(t, x) = x + \sum_{i=0}^d \int_0^t V_i I_N(X(s, x)) \circ dB^i(s).$$

Where $x \in \mathbb{R}^N$ and $V_0, \dots, V_d$ are tangent vector fields on $\mathbb{R}^N$ whose coefficients belong to $C_b^\infty(\mathbb{R}^N; \mathbb{R}^N)$, i.e. $V_i I_N = ((V_i I_N)^1 \dots (V_i I_N)^N) \in C_b^\infty(\mathbb{R}^N; \mathbb{R}^N)$ and the equality

$$(2) \quad V_i g(y) = \sum_{j=1}^N (V_i I_N)^j(y) \frac{\partial g}{\partial x_j}(y)$$

holds for all $i \in \{0, 1, \dots, d\}$, $g \in C^\infty(\mathbb{R}^N)$ and $y \in \mathbb{R}^N$. Where $\circ dB^i(t)$ denotes the Stratonovich integral by $B^i(t)$. Furthermore, SDE (1) can be expressed in an alternative form by utilising the Itô integral $dB^i(t)$:

$$(3) \quad X(t, x) = x + \int_0^t \tilde{V}_0(X(s, x)) dt + \sum_{i=1}^d \int_0^t V_i I_N(X(s, x)) dB^i(s),$$

where

$$(4) \quad (\tilde{V}_0 I_N)^k = (V_0 I_N)^k + \frac{1}{2} \sum_{i=1}^d V_i (V_i I_N)^k$$

for all $k \in \{1, \dots, N\}$. This equality is called Itô–Stratonovich transformation.

Remark 1. Conventionally, a vector field V and the vector composed of coefficient functions of V are usually denoted by the same letter V . However, in this paper, as we see in the discussion above, the latter is strictly denoted as VI_N . This may seem redundant, but we dare to use this notation in this paper. This distinction contributes to the clear description of the Itô–Stratonovich transformation (4) introduced immediately above, as well as Definitions 6, 8, 9 and 10, which are to be seen in Section 2 below.

1.2.2. Weak approximation of SDEs. Let $X(t, x)$ be a stochastic process defined by (1) and f be a $\mathbb{R}$ -valued function defined on $\mathbb{R}^N$. The numerical calculation of $E[f(X(T, x))]$ is referred to as the weak approximation of SDE (1), and it has been the focus of considerable research due to its significance in practical applications [9, 15].

1.2.3. Simulation method. Among the weak approximation methods for SDE, the one relevant to this paper is the simulation method. The weak approximation of SDE (1) by the simulation method is performed by the following procedure.

Let $\Delta = \{0 = t_0 < t_1 < \dots < t_n = T\}$ be a partition of the interval $[0, T]$. As usual, we define $\sharp\Delta = n$, $\Delta_k = t_k - t_{k-1}$ for $k \in \{1, \dots, \sharp\Delta\}$ and $|\Delta| = \max\{t_{i+1} - t_i \mid 0 \leq i \leq \sharp\Delta - 1\}$. We construct a set of random variables $\{X^{(\Delta)}(t_i, x)\}_{i=0}^{\sharp\Delta}$ that approximates $\{X(t, x)\}_{0 \leq t \leq T}$. The pair of the partition Δ of $[0, T]$ and this set of random variables $(\Delta, \{X^{(\Delta)}(t_i, x)\}_{i=0}^{\sharp\Delta})$ is called the discretisation of $X(t, x)$.

It should be noted that $X^{(\Delta)}(T, x)$ is an $\mathbb{R}^N$ -valued function defined over a finite-dimensional domain. If we denote the dimension of this domain by $D(\Delta)$, then we have a map $X^{(\Delta)}(T, x)(\cdot) : \mathbb{R}^{D(\Delta)} \rightarrow \mathbb{R}^N$. Finally, the numerical integration $E[f(X^{(\Delta)}(T, x))]$ is performed. This calculation is notorious as high-dimensional numerical integration and we have to resort to Monte Carlo or quasi-Monte Carlo methods.

1.2.4. Order of discretisation. As described above, the weak approximation calculation by the simulation method is performed through two stages of discretisation and numerical integration, with approximation errors occurring at each of these stages. The approximation error generated in the former stage is referred to as the discretisation error, while the error generated in the latter stage is referred to as the integration error. This paper deals only with the former type of error and does not consider the latter. This matter is discussed briefly in subsection 3.5.3 below.

Definition 1. $\{X^{(\Delta)}(t_i, x)\}_{i=0}^{\sharp\Delta}$ is defined to be a p -th order discretisation or a discretisation of order p if there exists a positive C_p and for all $n \in \mathbb{N}$ a partition Δ of $[0, T]$ such that $\sharp\Delta = n$ exists and the following inequality

$$\left| E[f(X(T, x))] - E[f(X^{(\Delta)}(T, x))] \right| \leq C_p (\sharp\Delta)^{-p}$$

holds.

Note that this definition is made using $\sharp\Delta$, rather than $|\Delta|$. This is because we are concerned with the trade-off between computational accuracy and computational load. Henceforth, what is referred to as high-order discretisation in this paper refers to discretisation of 2nd order or higher.

Henceforth, what is referred to as high-order discretisation in this paper refers to discretisation of 2nd order or higher.

1.3. Discretisation methods. We introduce some examples of discretisation methods. For a tangent vector field V on $\mathbb{R}^N$ and $x \in \mathbb{R}^N$, $\exp(V)x$ denotes $z(1)$ where $z(t)$ is the solution of the following ODE:

$$z(0) = x, \quad \frac{dz(t)}{dt} = V I_N(z(t)).$$

We remark that $\exp(tV)x = z(t)$ and $(d/dt)(f(\exp(tV)x)) = Vf(\exp(tV)x)$ hold. We will refer to $\exp(V)(\exp(W)x)$ as $\exp(V) \circ \exp(W)x$.

1.3.1. ODE solver, ODE integrator. The algorithm or method for numerically computing $\exp(V)x$ is called the ODE solver or ODE integrator.

1.3.2. Euler–Maruyama [21, 15]. The Euler–Maruyama discretisation $\{X^{(\text{EM}, \Delta)}(t_i, x)\}_{i=0}^{\sharp \Delta}$ is the most widely recognised technique for discretisation of SDEs.

Definition 2. The Euler–Maruyama discretisation of SDE (1) is defined as follows:

$$\begin{aligned} X^{(\text{EM}, \Delta)}(t_0, x) &= x \\ (5) \quad X^{(\text{EM}, \Delta)}(t_k, x) &= X^{(\text{EM}, \Delta)}(t_{k-1}, x) + \Delta_k \tilde{V}_0 I_N(X^{(\text{EM}, \Delta)}(t_{k-1}, x)) \\ &\quad + \sqrt{\Delta_k} \sum_{i=1}^d V_i I_N(X^{(\text{EM}, \Delta)}(t_{k-1}, x)) \eta_k^i \end{aligned}$$

where $\{\eta_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \sharp \Delta}}$ is a family of independent random variables with the standard normal distribution.

For the Euler–Maruyama discretisation, $D(\Delta) = d \times \sharp \Delta$. Under certain conditions, the Euler–Maruyama discretisation achieves 1st order accuracy [15].

1.3.3. Cubature 3 [20, 29].

Definition 3. Cubature 3 discretisation of SDE (1) is defined as follows:

$$\begin{aligned} X^{(\text{cub3}, \Delta)}(0, x) &= x \\ (6) \quad X^{(\text{cub3}, \Delta)}(t_k, x) &= \exp \left(\Delta_k V_0 + \sqrt{\Delta_k} \sum_{i=1}^d \eta_k^i V_i \right) X^{(\text{cub3}, \Delta)}(t_{k-1}, x) \end{aligned}$$

where $\{\eta_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \sharp \Delta}}$ is a family of independent random variables with the standard normal distribution.

According to (6), to find $X^{(\text{cub3}, \Delta)}(t_k, x)$, one needs to start from $X^{(\text{cub3}, \Delta)}(t_{k-1}, x)$ and proceed along the vector field $\Delta_k V_0 + \sqrt{\Delta_k} \sum_{i=1}^d \eta_k^i V_i$ for time 1.

This method also achieves 1st order accuracy and $D(\Delta) = d \times \sharp \Delta$ [29].

1.3.4. *High-order method I* [29]. The method presented below is a high-order discretisation method for weak approximation, possessing properties similar to those of the explicit Runge–Kutta method. Specifically, it can be implemented through iterations of function applications and linear combination operations. Currently, only two high-order discretisation methods with this property are known. These are the two methods described in this subsection and the following subsection 1.3.5

Definition 4. Let $\{\eta_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ be a family of independent random variables with the standard normal distribution and $\{\Lambda_k\}_{1 \leq k \leq \#\Delta}$ be a family of independent random variables such that $P(\Lambda_k = \pm 1) = 1/2$ for all k . Then a discretisation $\{X^{(\text{NV}, \Delta)}(t_k, x)\}_{0 \leq k \leq \#\Delta}$ of the SDE (1) is defined as follows:

(7)

$$\begin{aligned} X^{(\text{NV}, \Delta)}(0, x) &= x \\ X^{(\text{NV}, \Delta)}(t_k, x) &= \begin{cases} \exp\left(\frac{\Delta_k}{2} V_0\right) \left(\overrightarrow{\prod}_{i=1, \dots, d} \circ \exp\left(\sqrt{\Delta_k} \eta_k^i V_i\right) \right) \circ \exp\left(\frac{\Delta_k}{2} V_0\right) X^{(\text{NV}, \Delta)}(t_{k-1}, x) & \text{if } \Lambda_k = 1, \\ \exp\left(\frac{\Delta_k}{2} V_0\right) \left(\overleftarrow{\prod}_{i=1, \dots, d} \circ \exp\left(\sqrt{\Delta_k} \eta_k^i V_i\right) \right) \circ \exp\left(\frac{\Delta_k}{2} V_0\right) X^{(\text{NV}, \Delta)}(t_{k-1}, x) & \text{if } \Lambda_k = -1, \end{cases} \end{aligned}$$

where $\overrightarrow{\prod}_{i=1, \dots, d} \circ A_i = \circ A_1 \circ A_2 \circ \dots \circ A_d$ and $\overleftarrow{\prod}_{i=1, \dots, d} \circ A_i = \circ A_d \circ A_{d-1} \circ \dots \circ A_1$.

$\{X^{(\text{NV}, \Delta)}(t_k, x)\}_{0 \leq k \leq \#\Delta}$ achieves 2nd order discretisation of SDE (1) under certain conditions. With a little ingenuity it is possible to set $D(\Delta) = (d+1) \times \#\Delta$ in this method [24].

As illustrated in Definition 3 and the subsequent discussion, $X^{(\text{NV}, \Delta)}(t_k, x)$ is derived by solving ODEs with $X^{(\text{NV}, \Delta)}(t_{k-1}, x)$ as starting point. The difference is that in this case, we have to solve $(d+2)$ ODEs one by one by joining their solution curves.

Let's describe this procedure in more detail. First, the random variables Λ_k and $\{\eta_k^i\}_{i=1}^d$ are drawn.

In the case $\Lambda_k = 1$, starting from $X^{(\text{NV}, \Delta)}(t_{k-1}, x)$, it moves along the vector field V_0 for a time $\Delta_k/2$. After this, it changes direction and moves along the vector field V_d for a time $\sqrt{\Delta_k} \eta_k^d$. Then, it changes direction again and moves along the vector field V_{d-1} for a time $\sqrt{\Delta_k} \eta_k^{d-1}$. This motion is repeated, and finally, after moving along the vector field V_1 for a time $\sqrt{\Delta_k} \eta_k^1$, it moves again along the vector field V_0 for a time $\Delta_k/2$, arriving at $X^{(\text{NV}, \Delta)}(t_k, x)$. Note that time can be negative here.

In the case $\Lambda_k = -1$, the procedure described above is performed with the order of the vector fields reversed. That is, starting from $X^{(\text{NV}, \Delta)}(t_{k-1}, x)$ and proceeding along V_0 for a time $\Delta_k/2$, then along the vector field V_1 for a time $\sqrt{\Delta_k} \eta_k^1$, it changes direction and proceeds along V_2 , and finally along V_d , before proceeding along V_0 for a time $\Delta_k/2$. The destination reached is $X^{(\text{NV}, \Delta)}(t_k, x)$.

1.3.5. High-order method II [23].

Definition 5. Let $\{\xi_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ and $\{\eta_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ be families of independent random variables with the standard normal distribution. Then a discretisation $\{X^{(\text{NN}, \Delta)}(t_k, x)\}_{0 \leq k \leq \#\Delta}$ of the SDE (1) is defined as follows:

$$(8) \quad \begin{aligned} X^{(\text{NN}, \Delta)}(0, x) &= 0 \\ X^{(\text{NN}, \Delta)}(t_k, x) &= \exp \left(c_1 \Delta_k V_0 + \sqrt{R_{11} \Delta_k} \sum_{i=1}^d \eta_k^i V_i \right) \\ &\quad \circ \exp \left(c_2 \Delta_k V_0 + \sqrt{\Delta_k} \sum_{i=1}^d \zeta_k^i V_i \right) X^{(\text{NN}, \Delta)}(t_{k-1}, x) \end{aligned}$$

where $u \geq 1/2$,

$$\begin{aligned} c_1 &= \mp \sqrt{(2u-1)/2}, \quad c_2 = 1 - c_1, \quad R_{11} = u, \\ R_{22} &= 1 + u \pm \sqrt{2(2u-1)}, \quad R_{12} = -u \mp \sqrt{(2u-1)/2} \end{aligned}$$

and

$$\zeta_k^i = \frac{R_{12}}{\sqrt{R_{11}}} \eta_k^i + \sqrt{R_{22} - \frac{R_{12}^2}{R_{11}}} \xi_k^i.$$

$\{X^{(\text{NN}, \Delta)}(t_k, x)\}_{0 \leq k \leq \#\Delta}$ also achieves 2nd order discretisation of SDE (1) under certain conditions and $D(\Delta) = 2d \times \#\Delta$.

1.3.6. Remark on cubature on Wiener space. The subsequent paragraph offers a brief exposition of the relationships between the three approximation methods previously introduced — Cubature 3 and the two high-order discretisation methods — and the method of cubature on Wiener space [20].

If we replace the standard normal random variables in Definitions 3, 4 and 5 with discrete random variables that are consistent with the normal distribution up to a certain order of moments, we obtain cubatures on Wiener space. In particular, by replacing the iid family $\{\eta_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ in Definition 3 by such an iid family $\{\tilde{\eta}_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ defined as $P(\tilde{\eta}_k^i = \pm 1) = 1/2$ we obtain a cubature on Wiener space of order 1. Similarly, if we replace the iid families $\{\eta_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ and $\{\xi_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ in Definitions 4 and 5 by such iid families $\{\tilde{\eta}_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ and $\{\tilde{\xi}_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \#\Delta}}$ that are defined by

$$(9) \quad \begin{aligned} P(\tilde{\eta}_k^i = \pm \sqrt{3}) &= P(\tilde{\xi}_k^i = \pm \sqrt{3}) = \frac{1}{6} \\ P(\tilde{\eta}_k^i = 0) &= P(\tilde{\xi}_k^i = 0) = \frac{2}{3} \end{aligned}$$

respectively, we obtain cubatures on Wiener space of order 2. For a more detailed explanation, see [20], [29] and [23].

2. HIGH-ORDER DEEP NEURAL SDE NETWORK

This section introduces the concept of high-order deep neural SDE networks.

2.1. ResNet as Euler scheme. The structure of ResNet can be regarded as an Euler approximation, which is a typical numerical solution method for ODEs [10, 6]. Let this first be explained below.

2.1.1. Simple DNN and ResNet.

Definition 6. A DNN is defined to be two finite sets of maps $\{F_k : \mathbb{R}^N \rightarrow \mathbb{R}^N\}_{k=1}^M$ and $\{G_k : \mathbb{R}^N \rightarrow \mathbb{R}^N\}_{k=1}^M$. Each F_k has a set of parameters $\{\alpha_i\}_{i=1}^{m_k}$ which are to be learned through training. The F_k is called as the k -th layer and M as the depth of the DNN. For maps E and F , $E \circ F$ denotes the composition of them, i.e. $E \circ F(x) = E(F(x))$. I_N denotes the identity map on $\mathbb{R}^N$.

Simple DNN: A DNN $\{(F_k, G_k)\}_{k=1}^M$ is defined to be simple DNN when $G_1 = F_1$ and $G_k = F_k \circ G_{k-1}$ for $k \in \{2, \dots, M\}$.

ResNet: A DNN $\{(F_k, G_k)\}_{k=1}^M$ is defined to be ResNet when $G_1 = I_N + F_1$ and $G_k = G_{k-1} + F_k \circ G_{k-1}$ for $k \in \{2, \dots, M\}$.

The term G_{k-1} on the right hand side of the definition of ResNet corresponds to the connection called skipped connection or residual connection.

2.1.2. Euler scheme and ResNet. We recall that the Euler approximation $\{x^{(\text{Euler}, \Delta)}(t_i, x_0)\}_{i=0}^{\#\Delta}$ with respect to a partition $\Delta = \{0 = t_0 < t_1 < \dots < t_{\#\Delta} = T\}$ of an $\mathbb{R}^N$ -valued curve $\{x(t)\}_{t \in [0, T]}$ defined by the ODE:

$$(10) \quad x(0) = x_0, \quad \frac{dx}{dt}(t) = V I_N(t, x(t)),$$

which is equivalent to the alternative form $x(t) = \exp(tV)x_0$ where V is a tangent vector field on $\mathbb{R}^N$, is defined as

$$(11) \quad \begin{aligned} & x^{(\text{Euler}, \Delta)}(t_i, x_0) \\ &= \begin{cases} x_0 & \text{if } i = 0 \\ x^{(\text{Euler}, \Delta)}(t_{i-1}, x_0) + \Delta_i V I_N(t_{i-1}, x^{(\text{Euler}, \Delta)}(t_{i-1}, x_0)) & \text{if } i \geq 1. \end{cases} \end{aligned}$$

If we place F_k in Definition 6 as $F_k(\cdot) = \Delta_i V I_N(t_k, \cdot)$ it is easy to see that the correspondence:

$$(12) \quad x^{(\text{Euler}, \Delta)}(t_k, x_0) = G_k(x_0)$$

applies. This means that the process of ResNet learning the parameter α_k corresponds to learning the vector field V at time t_k . The Euler approximation is a first-order approximation. With this in mind, we define ResNet as a first-order Net.

2.2. High-order approximation scheme for ODEs and neural net. In light of the observations made in subsection 2.1.2 regarding first-order approximation methods, we proceed to construct neural networks corresponding to high-order approximation methods. It is notable that the neural network corresponding to the explicit approximation method of the Runge–Kutta type is relatively straightforward to construct.

2.2.1. *5th order neural net.* From [5] we introduce an explicit 5th order Runge–Kutta type approximation.

Definition 7. We define an approximation $\{x^{(\text{RK5}, \Delta)}(t_i, x_0)\}_{i=0}^{\#\Delta}$ of the ODE (10) with respect to Δ as follows:

$$\begin{aligned}
 \text{For } i \in \{1, 2, \dots, 6\} \quad Y_i &= x^{(\text{RK5}, \Delta)}(t_{k-1}, x_0) + \Delta_k \sum_{j < i} a_{ij} Z_j \\
 Z_i &= V I_N(Y_i) \\
 x^{(\text{RK5}, \Delta)}(t_k, x_0) &= x^{(\text{RK5}, \Delta)}(t_{k-1}, x_0) + \Delta_k \sum_{j=1}^6 b_j Z_j
 \end{aligned}
 \tag{13}$$

where

$$\begin{aligned}
 a_{21} &= 2/5, \quad a_{31} = 11/64, \quad a_{32} = 5/64, \quad a_{43} = 1/2, \quad a_{51} = 3/64, \\
 a_{52} &= -15/64, \quad a_{53} = 3/8, \quad a_{54} = 9/16, \quad a_{62} = 5/7, \quad a_{63} = 6/7, \\
 a_{64} &= -12/7, \quad a_{65} = 8/7, \\
 a_{ij} &= 0 \quad \text{otherwise,} \\
 b &= \begin{pmatrix} \frac{7}{90} & 0 & \frac{32}{90} & \frac{12}{90} & \frac{32}{90} & \frac{7}{90} \end{pmatrix}.
 \end{aligned}$$

This method achieves a 5th order approximation.

From this definition, it can be seen that this approximation method has a similar form to the explicit Runge–Kutta type. That is, the method can be performed by repeating only two types of operations: the application of functions and the linear combination of the results. Furthermore, this sequence of operations does not include forward references. These features allow the method to be implemented as a neural network, as the definition and subsequent Figure 1 below show.

Definition 8 (RK5net). A DNN $\{(F_k, G_k)\}_{k=0}^{\#\Delta}$ is defined to be RK5net as $F_k(\cdot) = V I_N(t_k, \cdot)$,

$$\begin{aligned}
 H_1 &= I_N \\
 H_2 &= I_N + \Delta_k a_{21} F_k \\
 H_3 &= I_N + \Delta_k (a_{32} F_k \circ H_2 + (a_{31} + a_{21}) F_k) \\
 H_4 &= I_N + \Delta_k (a_{43} F_k \circ H_3 + a_{32} F_k \circ H_2 + (a_{31} + a_{21}) F_k) \\
 H_5 &= I_N + \Delta_k (a_{54} F_k \circ H_4 + (a_{53} + a_{43}) F_k \circ H_3 + (a_{52} + a_{32}) F_k \circ H_2 \\
 &\quad + (a_{51} + a_{31} + a_{21}) F_k) \\
 H_6 &= I_N + \Delta_k (a_{65} F_k \circ H_5 + (a_{64} + a_{54}) F_k \circ H_4 + (a_{63} + a_{53} + a_{43}) F_k \circ H_3 \\
 &\quad + (a_{62} + a_{52} + a_{32}) F_k \circ H_2 + (a_{51} + a_{31} + a_{21}) F_k),
 \end{aligned}
 \tag{14}$$

and

$$G_k = \left(I_N + \Delta_k \sum_{j=1}^6 H_j \right) \circ G_{k-1},
 \tag{15}$$

where constants $\{a_{ij}\}_{1 \leq i, j \leq 6}$ and $\{b_i\}_{i=1}^6$ are the same as those defined in Definition 7.

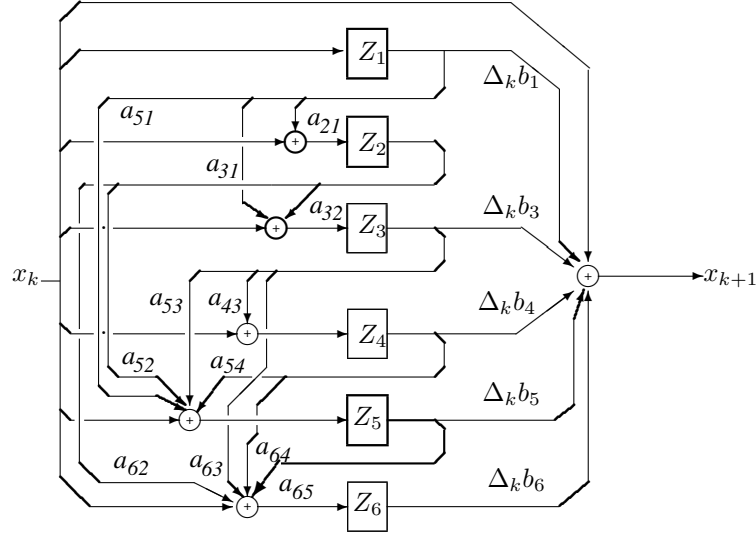


Figure 1. 5th order explicit Runge-Kutta type method

2.2.2. Orders of weak approximations for SDEs and those of approximation methods for ODEs. The weak approximation methods of SDEs introduced in Definitions 3, 4 and 5 all entail integral of some ODEs, i.e. numerical calculation of $\exp(tV)$, as part of their implementation. The following theorem, which concerns the relationship between the order of the weak approximation method for SDEs and the order of the approximation method for ODEs contained therein, is guaranteed to hold by Theorem 1.3 in [23].

Theorem 1. *The discretisation method for SDE (1), as defined in Definition 3, preserves order 1 if the ODEs it contains are solved using an ODE solver of order 3 or higher. Similarly, the methods defined in Definitions 4 and 5 preserve order 2 if the ODEs they contain are solved using an ODE solver of order 5 or higher.*

The claim of this theorem holds for more general p -order discrete approximation methods, including the three discretisation methods mentioned in the theorem, but to state the claim it is necessary to define the concept of an ODE-valued random variable, which is omitted here. See Theorem 1.3 in [23] for the exact claim.

2.3. High-order deep neural SDE network. The concept of a deep neural SDE network, which is a neural network that learns stochastic differential equations, is initially proposed in [4]. In this paper, a first-order method analogous to ResNet is utilised for the discretisation of SDEs.

In this section, we build on the preparation of the previous sections and put forward the concept of a high-order deep neural SDE network.

2.3.1. High-order deep neural SDE network: NVnet. $\{X^{(\text{NV}, \Delta)}(t_k, x)\}_{k=0}^{\#\Delta}$ in Definition 4 is implemented by series-connecting $(d + 2)$ ODE integrators. If each of these ODE integrator is implemented with the RK5net given in Definition 7, a neural network realising $\{X^{(\text{NV}, \Delta)}(t_k, x)\}_{k=0}^{\#\Delta}$ is obtained.

Definition 9 (NVnet). Let $\{\eta_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \sharp\Delta}}$ be a family of independent random variables with the standard normal distribution and $\{\Lambda_k\}_{1 \leq k \leq \sharp\Delta}$ be a family of independent random variables such that $P(\Lambda_k = \pm 1) = 1/2$ for all k . NVnet $\{(G_k, F_k)\}_{k=1}^{\sharp\Delta}$ is a DNN defined as:

- F_k , the k -th layer of NVnet is composed of $(d+2)$ ODE integrators, which are connected in series as follows:

$$(16) \quad \begin{aligned} & \exp\left(\frac{\Delta_k}{2} V_0(t_k, \cdot)\right) \left(\prod_{i=1, \dots, d}^{\rightarrow} \circ \exp\left(\sqrt{\Delta_k} \eta_k^i V_i(t_k, \cdot)\right) \right) \circ \exp\left(\frac{\Delta_k}{2} V_0(t_k, \cdot)\right) \\ & \hspace{15em} \text{if } \Lambda_k = 1, \\ & \exp\left(\frac{\Delta_k}{2} V_0(t_k, \cdot)\right) \left(\prod_{i=1, \dots, d}^{\leftarrow} \circ \exp\left(\sqrt{\Delta_k} \eta_k^i V_i(t_k, \cdot)\right) \right) \circ \exp\left(\frac{\Delta_k}{2} V_0(t_k, \cdot)\right) \\ & \hspace{15em} \text{if } \Lambda_k = -1. \end{aligned}$$

- All ODE integrators involved are implemented using RK5net.
- All $\sharp\Delta$ layers are connected in series, i.e. $G_1 = F_1$ and $G_k = F_k \circ G_{k-1}$ for $2 \leq k \leq \sharp\Delta$.

It follows from Theorem 1 that NVnet is a 2nd order neural SDE network. This network learns the vector fields V_i , which is equivalent to learning the stochastic process described by SDE (1).

2.3.2. High-order deep neural SDE network: NNnet. $\{X^{(\text{NN}, \Delta)}(t_k, x)\}_{0 \leq k \leq \sharp\Delta}$ is implemented by series connection of two ODE integrators from Definition 5. Thus, as in the case of NVnet, a neural network implementation of this can be obtained by using RK5net as an aid.

Definition 10 (NNnet). Let $\{\xi_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \sharp\Delta}}$, $\{\eta_k^i\}_{\substack{1 \leq i \leq d \\ 1 \leq k \leq \sharp\Delta}}$, u , c_1 , c_2 , R_{11} , R_{12} , R_{22} and ζ_k^i be the same as previously defined in Definition 5. NNnet is a deep neural network $\{(F_k, G_k)\}_{k=1}^{\sharp\Delta}$ defined as:

- F_k , the k -th layer of NNnet is composed of two ODE integrators, which are connected in series as follows:

$$(17) \quad \begin{aligned} & \exp\left(c_1 \Delta_k V_0 + \sqrt{R_{11} \Delta_k} \sum_{i=1}^d \eta_k^i V_i(t_k, \cdot)\right) \\ & \hspace{10em} \circ \exp\left(c_2 \Delta_k V_0 + \sqrt{\Delta_k} \sum_{i=1}^d \zeta_k^i V_i(t_k, \cdot)\right). \end{aligned}$$

- All ODE integrators involved are implemented using RK5net.
- All $\sharp\Delta$ layers are connected in series, i.e. $G_1 = F_1$ and $G_k = F_k \circ G_{k-1}$ for $2 \leq k \leq \sharp\Delta$.

Similarly to NVnet, NNnet is a 2nd order deep neural SDE network, for the same reasons. The vector fields V_i s are to be learnt.

3. NUMERICAL EXPERIMENT

The calculations are conducted using two types of deep neural SDE networks: NVnet, which is proposed in the preceding section, and ResNet for comparison. The networks are employed to compute the hedge martingale for an at-the-money (ATM) American vanilla put option.

3.1. American option pricing by deep neural SDE network. We let $(\Omega, \mathcal{F}, P)$ be a probability space with Brownian filtration $(\mathcal{F}_t)_{t \in [0, T]}$, $\mathcal{S}_T$ be the set of all $(\mathcal{F}_t)_{t \in [0, T]}$ -stopping times bounded by T and $(Z_t)_{t \in [0, T]}$ be the payoff process of some American option. It is widely acknowledged [2, 9] that the fair price of this American option is given by $\sup_{\tau \in \mathcal{S}_T} E[Z_\tau]$ and that it is notoriously challenging to calculate prices using the simulation method according to this formula. This difficulty arises from the fact that the formula involves optimisation over all stopping times bounded by T .

3.1.1. Rogers' dual algorithm. In [30], an expression free from stopping times for the appropriate price of the American option is given in accordance with the following theorem.

Theorem 2. *The price of an American option whose payoff process is given by $(Z_t)_{t \in [0, T]}$ is equal to*

$$(18) \quad \inf_{M \in H_0^1} E \left[\sup_{t \in [0, T]} (Z_t - M_t) \right]$$

where

$$H_0^1 = \left\{ M \mid (\mathcal{F}_t)_{t \in [0, T]} \text{-martingale s.t. } \sup_{t \in [0, T]} |M_t| \in L^1(P), M_0 = 0 \right\}.$$

This expression does not explore the space of stopping times; instead, it explores the space for martingales and identifies the lower bound of the maximum process of the difference between the payoff and the martingale. This is essentially a search for the optimal hedging process for the target American option.

3.1.2. Pricing process by learning machines. According to Theorem 2, the determination of the price of an American option by a learning machine amounts to the following slogan:

$$(19) \quad \text{Find } M^* \in \arg \inf_{M \in H_0^1} E \left[\sup_{t \in [0, T]} (Z_t - M_t) \right] \text{ by learning.}$$

This is carried out in two steps:

Step 1: Express the martingale $M_t \in H_0^1$ as

$$M_t = \int_0^t V_0^M I_N((X_u, M_u)_{0 \leq u \leq s}) ds + \sum_{j=1}^d \int_0^t V_j^M I_N((X_u, M_u)_{0 \leq u \leq s}) \circ dB^j(s)$$

implementing functions $V_0^M I_N, V_1^M I_N, \dots, V_d^M I_N$ in the form of simple MLP(=Multi Layer Perceptron) [8].

Step 2: Optimize all $V_j^M I_N$ s to minimize $E \left[\sup_{t \in [0, T]} (Z_t - M_t) \right]$ by using deep neural SDE network.

3.2. Important details of the implementation. This section discusses two technical issues that significantly influence the implementation of the proposed method. The first concerns the difficulty of incorporating the Itô–Stratonovich transformation (4) into a network architecture composed of MLPs. The second involves the challenge of obtaining an accurate maximum of a stochastic process under high-order discretisation.

3.2.1. Obstacle arising from the Itô–Stratonovich transformation. As noted in subsection 3.1.2, the vector fields $V_0^M, V_1^M, \dots, V_d^M$ are modelled using MLPs, whose parameters are learned during training. A central concern is ensuring that the resulting process M_t is a martingale. This concern arises because the networks (e.g., NVnet and NNnet) approximate the Stratonovich formulation of the dynamics, and hence, naively omitting the dt -term does not guarantee the martingale property.

To ensure correctness, one would need to realise the Itô–Stratonovich transformation within the neural network. Specifically, the transformation (4),

$$(V_0^M I_N)^k + \frac{1}{2} \sum_{i=1}^d V_i^M (V_i^M I_N)^k,$$

requires evaluating derivatives of vector fields (represented by MLPs) applied to their own coefficient functions, which are represented by the same MLPs. While it may be theoretically possible to construct a network that expresses this computation, it raises interesting challenges that fall outside the scope of this paper.

3.2.2. Measure-neutral implementation. A practical workaround to the above issue is available and avoids the aforementioned difficulties. Although the solution may appear counterintuitive, it proves effective in practice. The key idea is to design the implementation so that the process M_t remains centred across all sample paths, thereby satisfying the martingale condition in a distributional sense.

This is achieved as follows:

- Exclude V_0^M by setting $V_0^M = 0$.
- For each path ω_i , generate a provisional path $M'_t(\omega_i)$ using the network defined by the remaining vector fields $V_1^M, \dots, V_d^M$.
- Define the actual process $M_t(\omega_i)$ as a centred version:

$$(20) \quad M_t(\omega_i) = M'_t(\omega_i) - \frac{1}{K_{\text{BIN}}} \sum_{j=1}^{K_{\text{BIN}}} M'_t(\omega_j),$$

where K_{BIN} denotes the batch size used to estimate the expectation.

- Use the centred process M_t to evaluate the objective function in Rogers' dual formulation:

$$\frac{1}{K_{\text{BIN}}} \sum_{i=1}^{K_{\text{BIN}}} \sup_{t \in [0, T]} (Z_t(\omega_i) - M_t(\omega_i)).$$

This centring operation removes the drift component that would otherwise arise due to the absence of the V_0^M term, thereby enforcing a measure-neutral condition that approximates martingale behaviour.

The rationale for this approach is as follows. The provisional process M'_t generated by NVnet or NNnet is a drift-perturbed version of a martingale determined by the vector fields $V_1^M, \dots, V_d^M$, as dictated by the definitions of NVnet and NNnet. Thus, to construct a proper martingale M_t , one would ideally identify and cancel this drift by appropriately choosing V_0^M . The resulting process defined by $V_0^M, V_1^M, \dots, V_d^M$ would then coincide with a corrected version of M'_t , in which the drift has been eliminated. Therefore, the centring procedure described above—adjusting M'_t by subtracting its sample mean over paths—is a practical approximation to such a martingale M_t .

3.2.3. Simulation of the maximum process. It is well known that when trying to find $\sup_{t \in [0, T]} Y_t(\omega)$ by simulation with respect to a partition Δ , the naive method of taking $\max\{Y_i \mid i \in \{0, 1, \dots, \#\Delta\}\}$ as the value, is subject to such large errors that it becomes almost unusable as the width of the partition increases. This issue cannot be overlooked, particularly in our context. The NVnet and NNnet are founded upon high-order discrete approximations, which offer the primary benefit of enabling the partition width to be significantly expanded.

The well-known method of approximating $\sup_{t \in [t_{k-1}, t_k]} (Z_t(\omega) - M_t(\omega))$ using the Brownian bridge overcomes this difficulty. We regard $\sup_{t \in [t_{k-1}, t_k]} (Z_t(\omega) - M_t(\omega))$ as

$$(21) \quad \sup_{t \in [t_{k-1}, t_k]} \left\{ \sigma B_t \mid B_{t_{k-1}} = \frac{a}{\sigma} \quad \text{and} \quad B_{t_k} = \frac{b}{\sigma} \right\}$$

where $a = Z_{t_{k-1}}(\omega) - M_{t_{k-1}}(\omega)$ and $b = Z_{t_k}(\omega) - M_{t_k}(\omega)$. It is then necessary to find the value of the volatility σ . We draw about 10 sample paths from $Z_{t_{k-1}} - M_{t_{k-1}}$ and adopted their sample volatility. According to the well-known distribution function of pinned Brownian motion [13], we can calculate the cumulative density function $F_{\sigma B}$ of the pinned Brownian motion (21) and its inverse function $G_{\sigma B}$ as follows:

$$(22) \quad \begin{aligned} F_{\sigma B}(x) &= \int_{a \vee b}^x \frac{2(2y - b - a)}{\Delta_k} \exp\left(\frac{2(a - y)(y - b)}{\Delta_k}\right) dy \\ G_{\sigma B}(p) &= \frac{1}{2} \left(a + b + \sqrt{(a - b)^2 - 2\sigma^2 \Delta_k \log(1 - p)} \right). \end{aligned}$$

These can be employed to perform a simulation with a minimal computational load.

3.3. Specifications and parameters.

3.3.1. Computer and software used. Numerical experiments are conducted on a computational system equipped with an Intel i9-13900K CPU and an NVIDIA GTX4090 GPU. All numerical experiment in this article was programmed with TensorFlow 2, and parallel computation on GPU is employed.

3.3.2. Common parameters. The following specifications and parameters are commonly used in all numerical calculations in this study.

Each vector field V_j^M s is implemented by an MLP [8] consisting of two hidden layers, each with 32 nodes, and an output layer, also with 32 nodes. The ReLU activation function [8] is applied across all layers to ensure non-linearity and effective learning.

All numerical integrations included are proceeded by using Quasi-Monte Carlo method with $K_{\text{BIN}} = 5000$ sample points generated from the generalized Sobol'

sequence [28, 9]. As described below (in subsection 3.3.6), the parameters are updated at each evaluation in this experiment, i.e. EPOCH=1.

The result of Theorem 2 allows for the utilisation of the value of (18) itself as the LOSS function [8] for this learning.

3.3.3. Algorithm for updating parameters. The Adam optimiser [14, 8], which is provided by the TensorFlow library, is utilised with learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-07 and 2000 learning iterations are conducted to train MLPs.

Adam optimiser is an optimiser that updates parameters via dynamically scaling the update step based on moment estimation along each parameter's gradient vector.

Assuming we have a LOSS function which parameters are θ . Within Adam optimiser, we expect θ is updated according to its gradient w.r.t. LOSS function. Here, θ is the all parameters in MLPs. Adam optimiser differs from standard Stochastic Gradient Descent (SGD), Newton-type methods, and other non-parametric optimisers. SGD uses solely gradient information, and Newton's method uses gradient and Hessian information, however, they do not employ statistical moment to accelerate their parameter updating. Nevertheless, Adam optimiser takes advantage of 1st and 2nd statistical moment estimation. In the Adam optimiser, the moment can be regarded simply as weighted expectation of the mean and the variance of gradients. Since Adam optimiser avoids burdensome and memory-leaking Hessian computation, this feature results in Adam optimiser having a faster and well-behaved performance when it engages large scaled optimisation problems.

We briefly exhibit the mechanism of Adam optimiser below:

STEP1: 1st and 2nd Moment Initialisation:

$$\mathbf{m}_0 = 0, \quad \mathbf{v}_0 = 0, \quad t = 0$$

STEP2: Gradient Computation:

$$\mathbf{g}_t = \nabla_{\theta} \text{LOSS}(\theta_{t-1})$$

STEP3: 1st and 2nd Moment Estimation:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t \circ \mathbf{g}_t$$

STEP4: Bias Correction on 1st and 2nd Moment:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

STEP5: Parameter Updating:

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

where α is the learning rate, $\circ$ is Hadamard production and the term $\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$ is computed as element-wise.

All numerical integrations included are proceeded by using Quasi-Monte Carlo method with $K_{\text{BIN}} = 5000$ sample points generated from the generalized Sobol'

sequence [28, 9]. As described below (in subsection 3.3.6), the parameters are updated at each evaluation in this experiment, i.e. EPOC=1.

The result of Theorem 2 allows for the utilisation of the value of (18) itself as the LOSS function [8] for this learning.

3.3.4. Asset price models. The calculations are conducted for each case in which the risky asset follows two models: the Black–Scholes–Merton model [3] and the Heston stochastic volatility model [11].

The Black–Scholes–Merton model is obtained by setting $N = 1$, $X_t = S_t$, $V_0 I_1(y) = (\mu - \sigma^2/2)y$ and $V_1 I_1(y) = \sigma y$ in SDE (1). In the calculations with this model, we set $S_0 = 100$, $\mu = 0$ and $\sigma = 0.32$.

Furthermore, SDE (1) with $N = 2$, $X_t = {}^t(S_t \ U_t)$,

$$\begin{aligned} V_0 I_2 \left({}^t(y_1 \ y_2) \right) &= {}^t((\mu - y_2/2 - \rho\beta/4)y_1 \ \alpha(\theta - y_2) - \beta^2/4), \\ V_1 I_2 \left({}^t(y_1 \ y_2) \right) &= {}^t(y_1\sqrt{y_2} \ \rho\beta\sqrt{y_2}) \\ \text{and } V_2 I_2 \left({}^t(y_1 \ y_2) \right) &= {}^t(0 \ \beta\sqrt{(1 - \rho^2)y_2}) \end{aligned}$$

yields the Heston stochastic volatility model. Here, S_t represents the asset price at time t and U_t denotes its variance at that time. In the calculations with this model, we set $S_0 = 100$, $U_0 = 0.32$, $\mu = 0$, $\theta = 0.25$, $\alpha = 3.0$, $\rho = 0.3$ and $\beta = 0.4$.

Our target is an American put option on the asset whose price at t is S_t with a strike price of $K = 100$ and expiration date of $T = 1.0$. The payoff process Z_t is expressed as $Z_t = \max\{K - S_t, 0\}$.

3.3.5. Implementation of M_t . As described in subsection 3.1.2, each vector fields of M_t is implemented by an MLP. In this experiment, the MLPs are constrained to be of the form having (t, X_t, M_t) as input and output. In general, this restriction makes the space of martingales in which M_t moves smaller than H_0^1 . However, we ignore it here.

3.3.6. Procedure for one learning cycle. The following procedure is to be employed each time the MLP parameters are updated.

Step 1: Draw $\left\{ \left\{ X_{t_j}^{(\text{Alg}, \Delta)}(\omega_i) \right\}_{j=0}^{\Delta} \right\}_{i=1}^{K_{\text{BIN}}}$ and $\left\{ \left\{ M_{t_j}(\omega_i) \right\}_{j=0}^{\Delta} \right\}_{i=1}^{K_{\text{BIN}}}$. The former is generated in accordance with equation (5) when **Alg** is **EM** and in accordance with equation (7) when **Alg** is **NV**. The latter is generated in accordance with equation (20) by ResNet if **Alg** is **EM** or by NVnet if **Alg** is **NV**, following the procedure outlined in subsection 3.2.2.

Step 2: Approximate $E \left[\sup_{t \in [0, T]} (Z_t - M_t) \right]$ by the sample mean

$$(23) \quad \frac{1}{K_{\text{BIN}}} \sum_{i=1}^{K_{\text{BIN}}} \sup_{t \in [0, T]} (Z_t(\omega_i) - M_t(\omega_i))$$

which is calculated from the samples drawn in the previous step. Subsequently, the parameters of the MLP are updated in order to reduce the value of the expression given by (23).

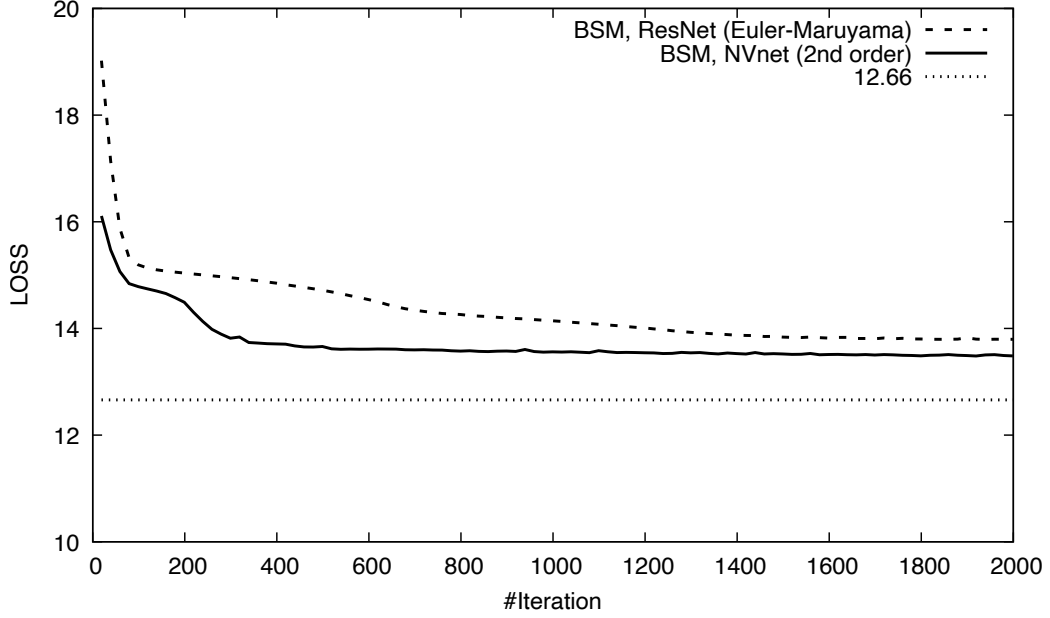


Figure 2. ResNet vs NVnet, Black–Scholes–Merton model

3.4. Results. Figures 2 and 3 show the results of the numerical experiments. In both figures, a comparison is made between learning with ResNet and learning with NVnet, one of the new architectures proposed in this paper. The vertical axis represents the LOSS and the horizontal axis represents the number of parameter updates, i.e. the number of learning iterations. Figure 2 shows the case where asset price follows the Black–Scholes–Merton model, while Figure 3 shows the case where it follows the Heston model. For ResNet, we set $\#\Delta = 1024$ and $\Delta_1 = \dots = \Delta_{1024} = 1/1024$; for NVnet $\#\Delta = 4$ and $\Delta_1 = \dots = \Delta_4 = 1/4$.

3.4.1. Black–Scholes–Merton model case. As demonstrated in Figure 2, the ResNet combined with the Euler–Maruyama case, i.e. the 1st order case, requires over 1500 learning iterations before the improvement due to learning becomes indistinguishable, whereas the NVnet combined with the high-order discretization case, i.e. the 2nd order case, achieves the same outcome after 250 learning iterations. It can also be seen that the optimisation results obtained by learning are significantly better with NVnet than with ResNet, i.e. a lower LOSS is achieved.

If the price of the underlying asset follows the Black-Scholes-Merton model, there is another well-known method of calculating the price of American options using recombining binary trees [12]. The price calculated by this method is 12.66. Figure 2 also shows prices calculated using this method for reference. It should be recalled that, as noted at the end of subsection 3.3.2, in these calculations, the LOSS coincides with the price itself that is being sought.

3.4.2. Heston model case. For the case of the Heston model, Figure 3 shows that even with 2000 learning iterations, both ResNet and NVnet still improve slightly through learning. Otherwise, the trend is the same as for the Black-Scholes-Merton

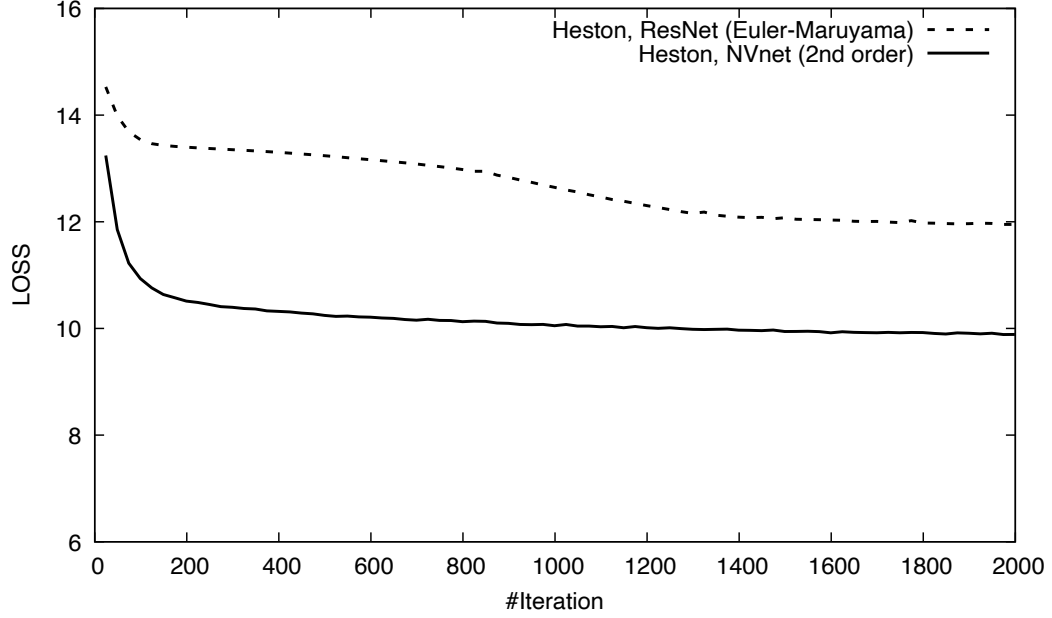


Figure 3. ResNet vs NVnet, Heston model

model described above, but the difference between ResNet and NVnet is even more pronounced. In particular, the difference in learning speed—i.e. the rate of LOSS reduction—is even greater: NVnet achieves an optimisation result in just 50 learning iterations—something that ResNet is unable to reach even after extensive training.

3.5. Discussions. This series of numerical experiments shows that the concept of high-order deep neural SDE networks, proposed from a purely mathematical point of view, and its enabling architecture, NVnet, are indeed effective in learning martingales. It performs significantly better than the first-order network architecture, ResNet, both in terms of the level of optimisation and the speed of learning.

The objective of this numerical experiment is to ascertain whether the optimisation process occurs as predicted. To this end, the implementation utilises libraries such as TensorFlow in their current state and without additional modifications, and crucial evaluations from an engineering perspective, including the assessment of the accuracy of the optimisation and the speed of learning, have not yet been conducted.

3.5.1. Immediate tasks. In particular, the following two issues are immediately obvious and need to be investigated further: (1) that in the case of the Black–Scholes–Merton model, the values obtained from the optimisation do not seem to reach those obtained from the binomial tree. (2) in the case of the Heston model, the number of training iterations does not seem to be sufficient, as described in subsection 3.4.2. For the first one, it is assumed that the optimisation was not sufficient because it was implemented to work for the time being, without sufficiently investigating

the implementation method of the vector field. For the second, it is due to excessive memory consumption, also caused by the makeshift implementation, and it is essential to examine the design from an engineering perspective next time.

3.5.2. NNnet. It is worth implementing NNnet and performing numerical verification in the same way as we do with NVnet. This is because, although both NVnet and NNnet are second-order SDE networks, their structures are completely different, and if they perform as well as NVnet, it will confirm the usefulness of the approach of learning martingales with higher-order NNs. We are currently in the process of implementing NNnet.

3.5.3. Monte Carlo and quasi-Monte Carlo free algorithms. As stated in subsection 1.2.4, the study did not take into account any integral error. This is because, as far as the present numerical experiments are concerned, the option pricing calculations under the two asset models treated here have already been dealt with in previous studies [29, 23, 24] and the integral errors are found to be almost negligible when calculated using these numbers of samples with quasi-Monte Carlo.

In general, however, it is worth considering algorithms that are free of Monte Carlo and quasi-Monte Carlo. This is because it is no longer possible to rely on these methods when it comes to the reproducibility and explainability of calculations as well as the quality assurance of results. Strictly speaking, quasi-Monte Carlo is reproducible, but in other respects it is not guaranteed in practice, just as Monte Carlo is not.

The high-order recombination measure method, on which the authors of this study and others have been working in recent years, is considered promising in this respect. The high-order recombination measure method is first proposed in [19]. The subsequent studies [26], [25] and [27] investigate the development of algorithms to implement this method, and its application to the numerical computation of weak approximations of SDEs in finance is demonstrated, thus underlining its practical significance.

REFERENCES

- [1] Shunichi Amari, *A theory of adaptive pattern classifiers*, IEEE Transactions on Electronic Computers **EC-16** (1967), no. 3, 299–307.
- [2] Tomas Björk, *Arbitrage theory in continuous time*, Oxford University Press, 12 2019.
- [3] F. Black and M. Scholes, *The Pricing of Options and Corporate Liabilities*, Journal of Political Economy **81** (1973), 637–59.
- [4] Hans Buehler, Lukas Gonon, Josef Teichmann, and Ben Wood, *Deep hedging*, Quantitative Finance **19** (2019), no. 8, 1271–1291.
- [5] John C. Butcher, *The Numerical Analysis of Ordinary Differential Equations*, John Wiley & Sons, Chichester, 1987.
- [6] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud, *Neural ordinary differential equations*, Proceedings of the 32nd International Conference on Neural Information Processing Systems (Red Hook, NY, USA), NIPS’18, Curran Associates Inc., 2018, p. 6572–6583.
- [7] Kunihiko Fukushima, *Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position*, Biological Cybernetics **36** (1980), 193–202.
- [8] Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, second ed., O’Reilly, 1005 Gravenstein Hwy N, Sebastopol, CA 95472, USA, 2019.
- [9] Paul Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer Verlag, New York, 2004.

- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, *Deep residual learning for image recognition*, 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [11] Steven L. Heston, *A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options*, The Review of Financial Studies **6** (1993), 327–343.
- [12] John Hull, *Options, Futures, and Other Derivatives*, Prentice Hall, Upper Saddle River, NJ, 2000.
- [13] Nobuyuki Ikeda and Shinzo Watanabe, *Stochastic differential equations and diffusion processes*, North Holland/Kodansha, 1981.
- [14] Diederik P. Kingma and Jimmy Ba, *Adam: A method for stochastic optimization*, CoRR **abs/1412.6980** (2014).
- [15] Peter E. Kloeden and Eckhard Platen, *Numerical Solution of Stochastic Differential Equations*, Springer Verlag, Berlin, 1999.
- [16] Shigeo Kusuoka, *Approximation of Expectation of Diffusion Process and Mathematical Finance*, Advanced Studies in Pure Mathematics, Proceedings of Final Taniguchi Symposium, Nara 1998 (T. Sunada, ed.), vol. 31, 2001, pp. 147–165.
- [17] ———, *Malliavin Calculus Revisited*, Journal of Mathematical Sciences The University of Tokyo **10** (2003), 261–277.
- [18] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, *Backpropagation applied to handwritten zip code recognition*, Neural Computation **1** (1989), no. 4, 541–551.
- [19] Christian Litterer and Terry Lyons, *High order recombination and an application to cubature on wiener space*, Ann. Appl. Probab. **22** (2012), no. 4, 1301–1327.
- [20] Terry Lyons and Nicolas Victoir, *Cubature on Wiener Space*, Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences **460** (2004), 169–198.
- [21] Gisirô Maruyama, *Continuous markov processes and stochastic equations*, Rendiconti del Circolo Matematico di Palermo **4** (1955), 48–90.
- [22] GN Milshtejn, *Approximate integration of stochastic differential equations*, Theory of Probability & Its Applications **19** (1975), no. 3, 557–562.
- [23] Mariko Ninomiya and Syoiti Ninomiya, *A new higher-order weak approximation scheme for stochastic differential equations and the Runge—Kutta method*, Finance and Stochastics **13** (2009), 415–443, 10.1007/s00780-009-0101-4.
- [24] Syoiti Ninomiya and Yuji Shinozaki, *Higher-order Discretization Methods of Forward-backward SDEs Using KLVN-scheme and Their Applications to XVA Pricing*, Applied Mathematical Finance **26** (2019), no. 3, 257–292.
- [25] ———, *On implementation of high-order recombination and its application to weak approximations of stochastic differential equations*, Proceedings of the NFA 29th Annual Conference, 2021.
- [26] ———, *Patch dividing algorithms for high-order recombination and its application to weak approximations of stochastic differential equations*, 2023, Presented at Workshop on Probabilistic methods, Signatures, Cubature and Geometry, 2023/01/09-11 (York University, York, UK).
- [27] ———, *A high-order recombination algorithm for weak approximation of stochastic differential equations*, 2025, arXiv:2504.19717.
- [28] Syoiti Ninomiya and Shu Tezuka, *Toward real-time pricing of complex financial derivatives*, Applied Mathematical Finance **3** (1996), 1–20.
- [29] Syoiti Ninomiya and Nicolas Victoir, *Weak Approximation of Stochastic Differential Equations and Application to Derivative Pricing*, Applied Mathematical Finance **15** (2008), no. 2, 107–121.
- [30] L. C. G. Rogers, *Monte carlo valuation of american options*, Mathematical Finance **12** (2002), no. 3, 271–286.

²DEPARTMENT OF ENGINEERING AND ECONOMICS, SCHOOL OF ENGINEERING, INSTITUTE OF SCIENCE TOKYO, 2-12-1 OOKAYAMA, MEGURO-KU, TOKYO 152-8552 JAPAN

¹DEPARTMENT OF MATHEMATICS, SCHOOL OF SCIENCE, INSTITUTE OF SCIENCE TOKYO, 2-12-1 OOKAYAMA, MEGURO-KU, TOKYO 152-8551 JAPAN
 Email address: syoiti.ninomiya@gmail.com